

CENTRO UNIVERSITÁRIO UNIFECAP

KEVIN MAIKON CAETANO DE ANCRAGE SANTOS

**SISTEMA AUTOMOTIVO: GESTÃO DE ESTOQUE DE
VEÍCULOS**

TABOÃO DA SERRA

2026

KEVIN MAIKON CAETANO DE ANCRAGE SANTOS

SISTEMA AUTOMOTIVO: GESTÃO DE ESTOQUE DE VEÍCULOS

Trabalho acadêmico apresentado ao Centro Universitário UNIFECAP como parte das atividades avaliativas da disciplina de Programação Orientada a Objetos.

TABOÃO DA SERRA

2026

1. INTRODUÇÃO

O presente trabalho tem como objetivo apresentar o desenvolvimento de um sistema automotivo para gestão de estoque de veículos, utilizando os conceitos de Programação Orientada a Objetos, persistência de dados e desenvolvimento de aplicações web com Java e Spring Boot.

O sistema foi desenvolvido para auxiliar concessionárias, vendedores e administradores no controle de veículos disponíveis em estoque. Por meio da aplicação, é possível cadastrar veículos, marcas e modelos, consultar informações, atualizar dados, excluir registros e aplicar filtros de pesquisa.

A proposta busca resolver um problema comum em empresas do setor automotivo: a dificuldade de manter informações organizadas, atualizadas e acessíveis sobre os veículos disponíveis para venda. Com o sistema, os dados ficam centralizados em um banco de dados MySQL e podem ser acessados por meio de uma API REST e de uma interface web simples.

2. OBJETIVO DO PROJETO

O objetivo principal do projeto é desenvolver um sistema CRUD orientado a objetos para gerenciamento de estoque de veículos.

Os objetivos específicos são:

- Permitir o cadastro de veículos.
- Permitir a consulta de veículos cadastrados.
- Permitir a atualização de informações dos veículos.
- Permitir a exclusão de veículos.
- Permitir filtros por marca, modelo, ano, faixa de preço e status.
- Permitir o cadastro de marcas.
- Permitir o cadastro de modelos associados às marcas.
- Utilizar banco de dados MySQL para persistência das informações.
- Aplicar conceitos de Programação Orientada a Objetos.
- Utilizar Java, Spring Boot, Spring Data JPA e Maven no desenvolvimento.

3. DESCRIÇÃO DO PROBLEMA

Muitas concessionárias e lojas de veículos ainda enfrentam dificuldades para organizar seus estoques. Quando as informações são controladas manualmente, em planilhas ou em sistemas pouco integrados, podem ocorrer erros, perda de dados, atraso na

atualização de informações e dificuldade para localizar veículos específicos.

Esses problemas afetam diretamente vendedores, clientes e gestores. Os vendedores podem perder tempo procurando informações sobre veículos disponíveis. Os clientes podem não encontrar rapidamente o modelo desejado. Já os gestores podem ter dificuldade para acompanhar o estoque, analisar disponibilidade e tomar decisões estratégicas.

Dessa forma, o sistema proposto busca centralizar as informações dos veículos em uma aplicação organizada, permitindo maior controle, agilidade e confiabilidade na gestão do estoque.

4. LEVANTAMENTO DE REQUISITOS

A seguir são apresentadas perguntas utilizadas para simular um levantamento de requisitos com possíveis usuários do sistema.

4.1 Perguntas e respostas simuladas

1. Qual é o principal problema enfrentado atualmente no controle de veículos?

Resposta: A falta de organização das informações dos veículos, pois os dados ficam espalhados ou desatualizados.

2. Quais informações precisam ser cadastradas para cada veículo?

Resposta: Modelo, marca, ano, cor, preço, quilometragem e status de disponibilidade.

3. Quais status um veículo pode possuir?

Resposta: Disponível, vendido ou reservado.

4. O sistema precisa permitir consulta de veículos?

Resposta: Sim, é necessário listar todos os veículos cadastrados e também buscar um veículo específico por ID.

5. O sistema deve permitir filtros?

Resposta: Sim, deve permitir filtros por marca, modelo, ano, faixa de preço e status.

6. É necessário atualizar dados de veículos?

Resposta: Sim, principalmente preço, quilometragem e status.

7. O sistema deve permitir exclusão de veículos?

Resposta: Sim, veículos vendidos ou descontinuados podem ser removidos do estoque.

8. É necessário cadastrar marcas separadamente?

Resposta: Sim, para organizar melhor os modelos e relacioná-los às marcas.

9. É necessário cadastrar modelos separadamente?

Resposta: Sim, cada modelo deve estar associado a uma marca.

10. Quem utilizará o sistema?

Resposta: Vendedores, administradores e responsáveis pelo controle de estoque da

concessionária.

11. O sistema precisa armazenar os dados em banco?

Resposta: Sim, os dados devem ser persistidos em um banco MySQL.

12. O sistema precisa ter interface visual?

Resposta: Uma interface simples é útil para demonstração, mas a parte principal do sistema é a API REST.

5. REQUISITOS FUNCIONAIS

- RF01: O sistema deve permitir o cadastro de veículos.
- RF02: O sistema deve permitir a listagem de veículos.
- RF03: O sistema deve permitir a busca de veículo por ID.
- RF04: O sistema deve permitir a atualização de veículos.
- RF05: O sistema deve permitir a exclusão de veículos.
- RF06: O sistema deve permitir filtro por marca.
- RF07: O sistema deve permitir filtro por modelo.
- RF08: O sistema deve permitir filtro por ano.
- RF09: O sistema deve permitir filtro por faixa de preço.
- RF10: O sistema deve permitir filtro por status.
- RF11: O sistema deve permitir o cadastro de marcas.
- RF12: O sistema deve permitir o cadastro de modelos.
- RF13: O sistema deve associar modelos às suas respectivas marcas.
- RF14: O sistema deve validar os dados enviados pelo usuário.
- RF15: O sistema deve retornar mensagens adequadas em caso de erro.

6. REQUISITOS NÃO FUNCIONAIS

- RNF01: O sistema deve ser desenvolvido em Java 21.
- RNF02: O sistema deve utilizar Spring Boot.
- RNF03: O sistema deve utilizar Spring Data JPA para persistência.
- RNF04: O sistema deve utilizar banco de dados MySQL.
- RNF05: O sistema deve utilizar Maven para gerenciamento de dependências.
- RNF06: O sistema deve possuir organização em camadas.

- RNF07: O sistema deve ter código organizado e de fácil manutenção.
- RNF08: O sistema deve validar dados obrigatórios.
- RNF09: O sistema deve retornar respostas HTTP adequadas.
- RNF10: O sistema deve criar ou atualizar as tabelas automaticamente por meio do JPA/Hibernate.

7. ENTIDADES DO SISTEMA

7.1 Veículo

A entidade Veículo representa os automóveis cadastrados no estoque da concessionária. Seus principais atributos são id, modelo, marca, ano, cor, preço, quilometragem e status.

7.2 Marca

A entidade Marca representa as fabricantes dos veículos, como Honda, Toyota, Chevrolet, Jeep e Volkswagen. Essa entidade ajuda a organizar os modelos e permite associar modelos à marca correta.

7.3 Modelo

A entidade Modelo representa o nome comercial do veículo, como Civic, Corolla, Onix, Compass ou Golf. Cada modelo pertence a uma marca.

8. ARQUITETURA DO SISTEMA

O sistema foi organizado em camadas, seguindo boas práticas de desenvolvimento com Spring Boot. As principais camadas são Controller, Service, Repository, Entity, DTO, Exception e Config.

8.1 Controller

A camada controller é responsável por receber as requisições HTTP enviadas pelo usuário, navegador, Postman ou interface web.

8.2 Service

A camada service contém as regras de negócio da aplicação. Ela faz a ligação entre o controller e o repository.

8.3 Repository

A camada repository é responsável pela comunicação com o banco de dados. O projeto utiliza Spring Data JPA, que fornece métodos prontos para salvar, listar, buscar e excluir dados.

8.4 Entity

A camada entity contém as classes que representam as tabelas do banco de dados.

8.5 DTO

Os DTOs são usados para transportar dados entre a API e o cliente. Eles ajudam a separar os dados recebidos e devolvidos pela aplicação das entidades do banco.

8.6 Exception

A camada exception centraliza o tratamento de erros da aplicação, permitindo respostas padronizadas quando ocorre erro de validação, recurso não encontrado ou erro inesperado.

9. FLUXO DA APLICAÇÃO

O funcionamento da aplicação segue o seguinte fluxo: o usuário acessa a interface web ou envia uma requisição pelo Postman; a requisição chega ao controller; o controller chama o service; o service aplica as regras de negócio; o repository acessa o banco de dados MySQL; o resultado é retornado ao usuário.

Fluxo resumido: Interface Web ou Postman → Controller → Service → Repository → MySQL.

10. CONCEITOS DE PROGRAMAÇÃO ORIENTADA A OBJETOS APLICADOS

10.1 Classes e objetos

As classes representam elementos do sistema, como Veículo, Marca e Modelo. Cada veículo cadastrado no sistema é tratado como um objeto da classe Veículo.

10.2 Atributos

Os atributos armazenam as características dos objetos. No caso da classe Veículo, existem atributos como modelo, marca, ano, cor, preço, quilometragem e status.

10.3 Métodos

Os métodos representam comportamentos das classes. No projeto, as entidades possuem métodos para criação e atualização de dados.

10.4 Encapsulamento

O encapsulamento é aplicado por meio de atributos privados e métodos públicos de acesso. Isso protege os dados internos das classes e mantém maior controle sobre as alterações.

10.5 Enum

O enum StatusVeiculo define os valores permitidos para o status de um veículo, evitando que sejam cadastrados status inválidos. Os valores permitidos são DISPONIVEL, VENDIDO e RESERVADO.

10.6 Associação entre classes

A associação aparece na relação entre Modelo e Marca. Cada modelo pertence a uma marca, representando um relacionamento entre objetos.

10.7 Abstração

A abstração está presente principalmente nos repositories. O Spring Data JPA permite utilizar interfaces para acessar o banco sem escrever manualmente todos os comandos SQL.

10.8 Herança e interfaces

O projeto utiliza interfaces do Spring Data JPA, como JpaRepository. Os repositories herdam comportamentos prontos dessas interfaces, como salvar, buscar, listar e excluir registros.

11. JUSTIFICATIVA DAS TECNOLOGIAS UTILIZADAS

Java 21: foi escolhido por ser uma linguagem robusta, orientada a objetos e muito utilizada no mercado para desenvolvimento de sistemas corporativos.

Spring Boot: facilita a criação de aplicações web em Java, oferecendo configuração simplificada, servidor embutido e integração com diversas ferramentas.

Spring Data JPA: foi utilizado para facilitar a comunicação com o banco de dados e reduzir a necessidade de comandos SQL manuais.

MySQL: foi escolhido por ser um banco de dados relacional popular, gratuito e amplamente utilizado em aplicações comerciais e acadêmicas.

Maven: foi utilizado para gerenciar dependências, compilar o projeto e executar testes.

HTML, CSS e JavaScript: foram utilizados para criar uma interface web simples, permitindo demonstrar o funcionamento do sistema de forma visual.

12. ENDPOINTS PRINCIPAIS

12.1 Veículos

- POST /veiculos
- GET /veiculos
- GET /veiculos/{id}
- PUT /veiculos/{id}
- DELETE /veiculos/{id}
- GET /veiculos/filtro

12.2 Marcas

- POST /marcas
- GET /marcas
- GET /marcas/{id}
- PUT /marcas/{id}
- DELETE /marcas/{id}

12.3 Modelos

- POST /modelos
- GET /modelos
- GET /modelos/{id}
- PUT /modelos/{id}
- DELETE /modelos/{id}

13. EXEMPLO DE CADASTRO DE VEÍCULO

Exemplo de JSON utilizado para cadastrar um veículo:

```
{
  "modelo": "Civic",
  "marca": "Honda",
  "ano": 2022,
  "cor": "Prata",
  "preco": 135000.00,
  "quilometragem": 18000,
  "status": "DISPONIVEL"
}
```

14. INTERFACE WEB

Além da API REST, o projeto possui uma interface web simples para demonstração das principais funcionalidades do sistema. Essa interface permite cadastrar, listar, editar, excluir e filtrar veículos de forma visual pelo navegador.

Para acessar a interface, primeiro é necessário que a aplicação esteja em execução. Portanto, o usuário deve abrir o projeto, configurar o banco MySQL e iniciar o Spring Boot pelo terminal com o comando:

```
mvn spring-boot:run
```

Após a aplicação iniciar corretamente, será exibida no terminal uma mensagem informando que o servidor Tomcat foi iniciado na porta 8080. Somente depois disso a interface poderá ser acessada pelo navegador.

Em ambiente local, normalmente o endereço utilizado é:

```
http://localhost:8080/
```

Esse endereço funciona quando o sistema está rodando no mesmo computador em que o navegador está sendo utilizado. Caso a aplicação esteja sendo executada em outro ambiente, como máquina virtual, servidor remoto ou ambiente em nuvem, será necessário acessar o endereço correspondente ao ambiente onde o projeto estiver rodando.

A API REST também pode ser acessada diretamente em formato JSON. Por exemplo, após iniciar a aplicação, a listagem de veículos pode ser consultada pelo endpoint:

```
/veiculos
```

Em ambiente local, o endereço completo costuma ser:

```
http://localhost:8080/veiculos
```

Dessa forma, a interface web é utilizada para facilitar a visualização do sistema, enquanto os endpoints da API permitem testar as funcionalidades também por ferramentas como Postman ou Insomnia.

15. BANCO DE DADOS

O banco de dados utilizado é o MySQL. A aplicação utiliza JPA/Hibernate para criar e atualizar automaticamente as tabelas.

As principais tabelas são veículos, marcas e modelos.

Relacionamento principal: Marca 1:N Modelo. Isso significa que uma marca pode possuir vários modelos, mas cada modelo pertence a uma única marca.

16. TESTES E VALIDAÇÕES

O sistema possui validações nos dados enviados pelo usuário. Campos obrigatórios, valores mínimos e formatos incorretos são tratados pela aplicação.

Exemplos de validações: modelo obrigatório, marca obrigatória, ano obrigatório, preço maior que zero, quilometragem não negativa e status obrigatório.

Também existe tratamento de erros para casos como veículo não encontrado, marca não encontrada, modelo não encontrado, JSON inválido e erro de validação.

17. RESULTADOS OBTIDOS

Com o desenvolvimento do sistema, foi possível criar uma aplicação funcional para gestão de estoque de veículos. O sistema permite o cadastro completo de veículos, marcas e modelos, além de consultas, filtros, atualizações e exclusões.

A aplicação também demonstra a integração entre Java, Spring Boot e MySQL, aplicando conceitos de Programação Orientada a Objetos e organização em camadas.

18. CONCLUSÃO

O projeto desenvolvido atende ao objetivo proposto de criar um sistema automotivo para gestão de estoque de veículos. A aplicação permite controlar veículos cadastrados, organizar marcas e modelos, realizar filtros e manter os dados persistidos em banco MySQL.

Durante o desenvolvimento, foram aplicados conceitos importantes de Programação Orientada a Objetos, como classes, objetos, atributos, métodos, encapsulamento, enum, associação entre classes e uso de interfaces.

Além disso, o uso do Spring Boot facilitou a criação da API REST, enquanto o Spring Data JPA simplificou a integração com o banco de dados. A interface web simples contribui para demonstrar o funcionamento do sistema de forma visual.

Portanto, o sistema representa uma solução prática e organizada para o problema de controle de estoque de veículos em concessionárias.

19. REFERÊNCIAS

DEITEL, Paul; DEITEL, Harvey. **Java: Como Programar**. 10. ed. São Paulo: Pearson, 2016.

ORACLE. Java Documentation. Disponível em: <https://docs.oracle.com/en/java/>

SPRING. Spring Boot Documentation. Disponível em: <https://spring.io/projects/spring-boot>

SPRING. Spring Data JPA Documentation. Disponível em: <https://spring.io/projects/spring-data-jpa>

MYSQL. MySQL Documentation. Disponível em: <https://dev.mysql.com/doc/>